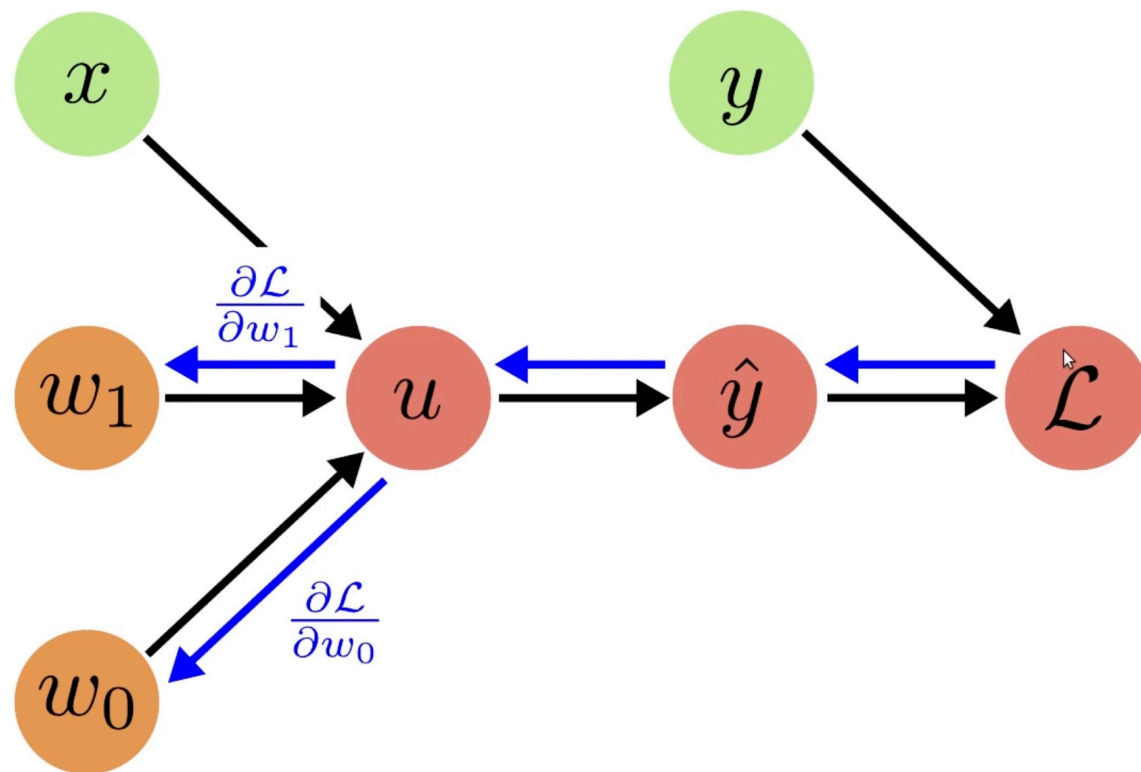


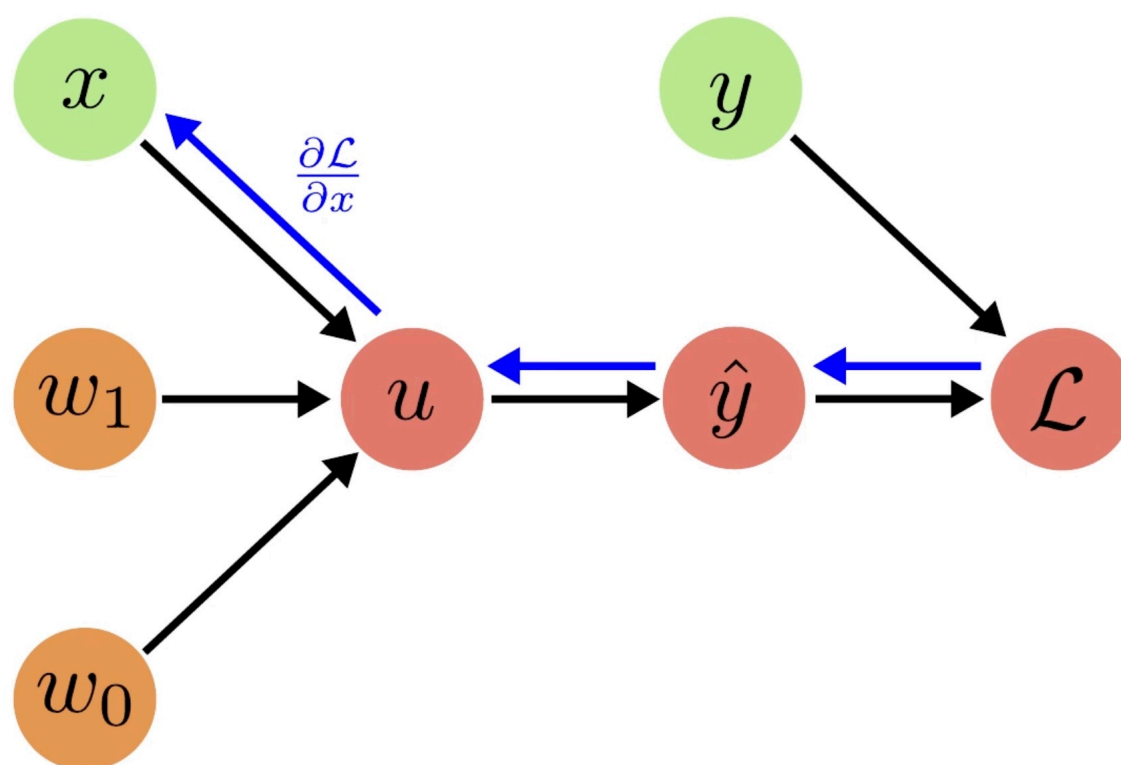
Lecture 12.1: Input Optimization

Weight vs Input optimization

Normally, we optimize weights for fixed inputs



In input optimization, we fix weights i.e take a pre-trained model & optimize inputs



This is what happens in Adversarial attacks & style transfer.

Adversarial Attacks on Image Classification

We take an image correctly classified by the model & add little bit noise to each pixel.

The new image looks same to humans as the noise is very small but the model classifies it wrongly very confidently.

L-BFGS Attack:

Given classifier $f: \mathbb{R}^m \rightarrow \{1 \dots L\}$

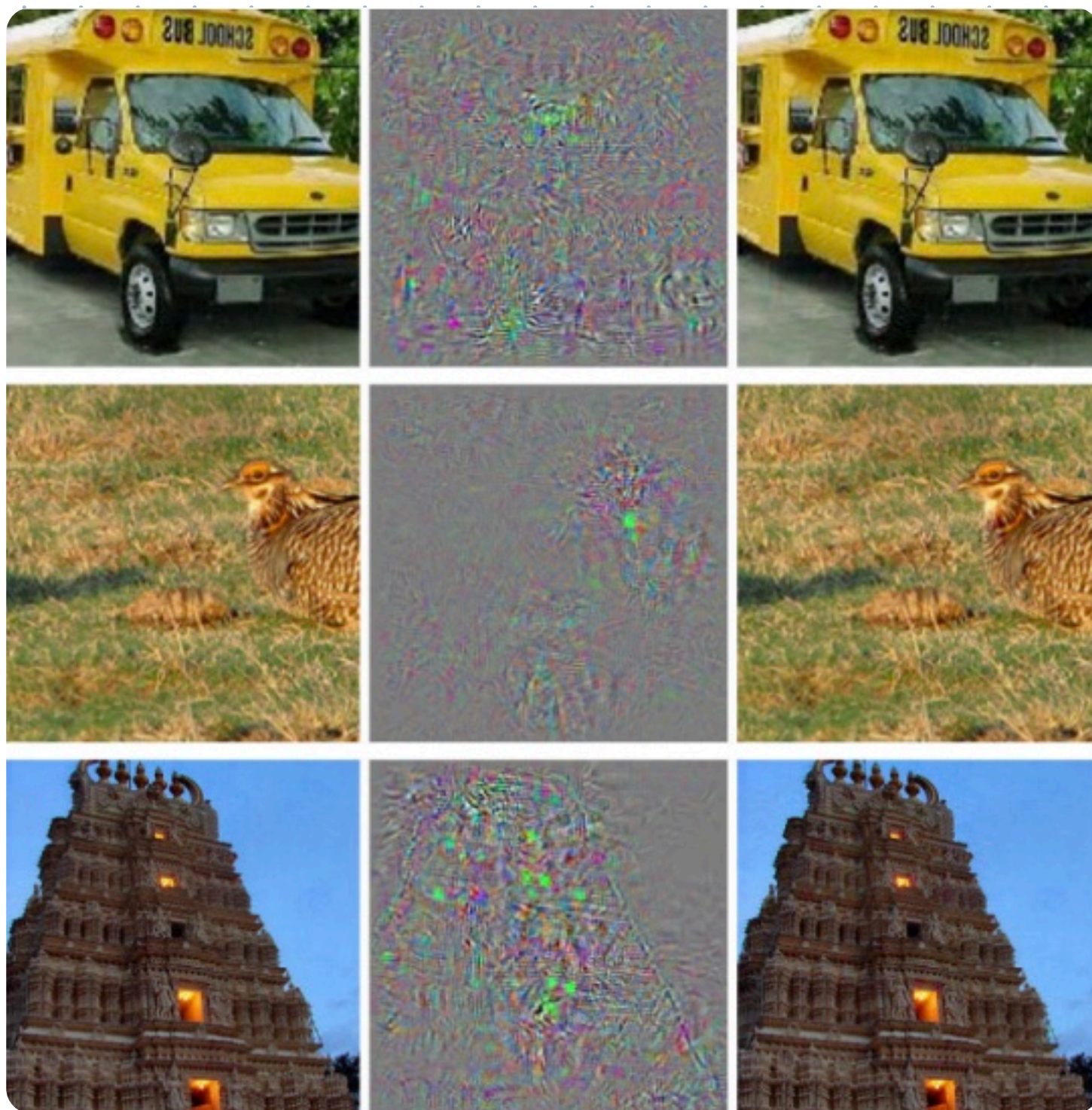
Find an adversarial example for image x :

$$x + \underset{\Delta x}{\operatorname{argmin}} \{ \|\Delta x\|_2 : f(x + \Delta x) = y_t \}$$

↳ choose a noise over a set of noises, whose value is small $\|\Delta x\|_2$ & the image ends being misclassified

where y_t is the target class

In short, we want to find the small perturbations such that when we add it to the input image & classify it we get class that we want.



This demonstrates how fragile NNs are.

This works for segmentation as well.

but is this really a problem? This assumes that we are able to manipulate every pixel of the input image.

Physical Adversarial Attacks



As there are many intermediate steps before image is seen by the model eg sensor adds noise, optics manipulates the image, all these transformations made the attack ineffective. Hence, their statement:

"Adversarial perturbation methods applied to **stop sign detection** only work in carefully chosen situations, and our preliminary experiment shows that we might **not need to worry** about it in many real circumstances, specifically with autonomous vehicles."

but as it turns out, it is a problem

Robust Adversarial Attacks

There are existence of robust adversarial examples in the physical world.

Maximize expectation over transformations $T(EOT)$

$$\operatorname{argmax}_{x'} \mathbb{E}_{t \sim \mathcal{T}} [\log P(y_t | t(x')) - \lambda \|t(x') - t(x)\|_2]$$

↳ minimize
log likelihood
of target class
 x_t (one we chose)

↳ minimize
distance b/w
transformed x' &
transformed x
image i.e transformation
is small

↳ A set of affine
or perspective
transformations

This shows that classifiers in the physical world can be attacked, by constructing examples that work regardless of intermediate step noise or viewpoint.

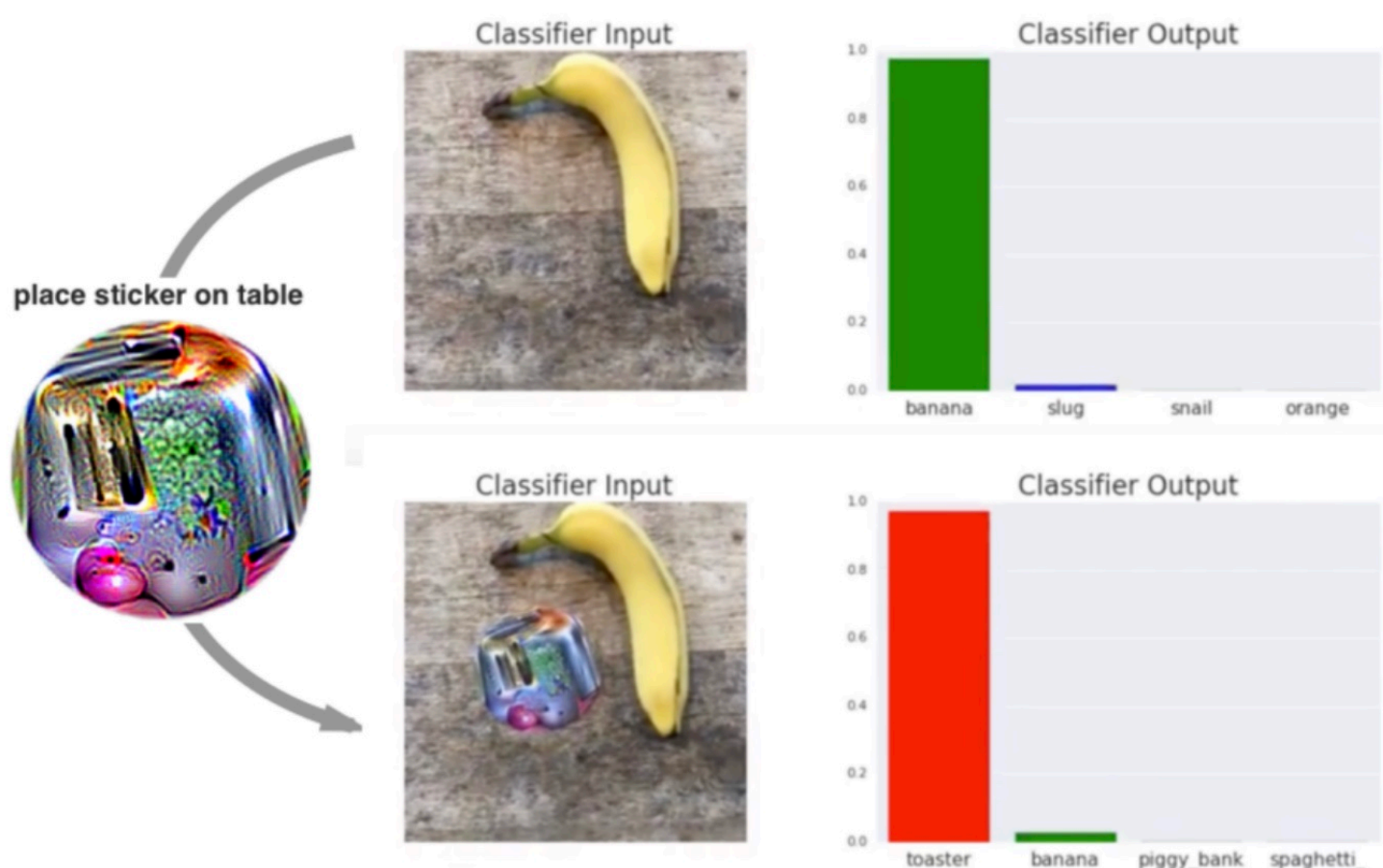
It is important to note that x' exists in the real-world, not a noise that is added before passing to the classifier.

Larger distributions $\uparrow$ require larger perturbations, it might become visible to human eye as well.



Adversarial patch attacks

We place a patch (that might arbitrary) to the input image.



When the patches are optimized for a specific network, it is called a White-Box

Attack. Generally, larger patches cause a higher disturbance of the predicted optical flow. However, even small patches disturb the prediction far beyond the attacked area.

Furthermore, a patch can be trained on a discrete set of networks (e.g. FlowNet2 and

Let $F(I, I')$ denote an **optical flow network** and $\mathcal{I}$ a dataset of frame pairs $\{(I, I')\}$.

Let $A(I, p, t, l)$ denote **image** I with **patch** p transformed by t inserted at location l .
Let $\mathcal{T}$ denote a distribution over affine 2D transformations.
Let $\mathcal{L}$ denote a (uniform) distribution over the image domain.

Goal: Find a patch $\hat{p}$ such that

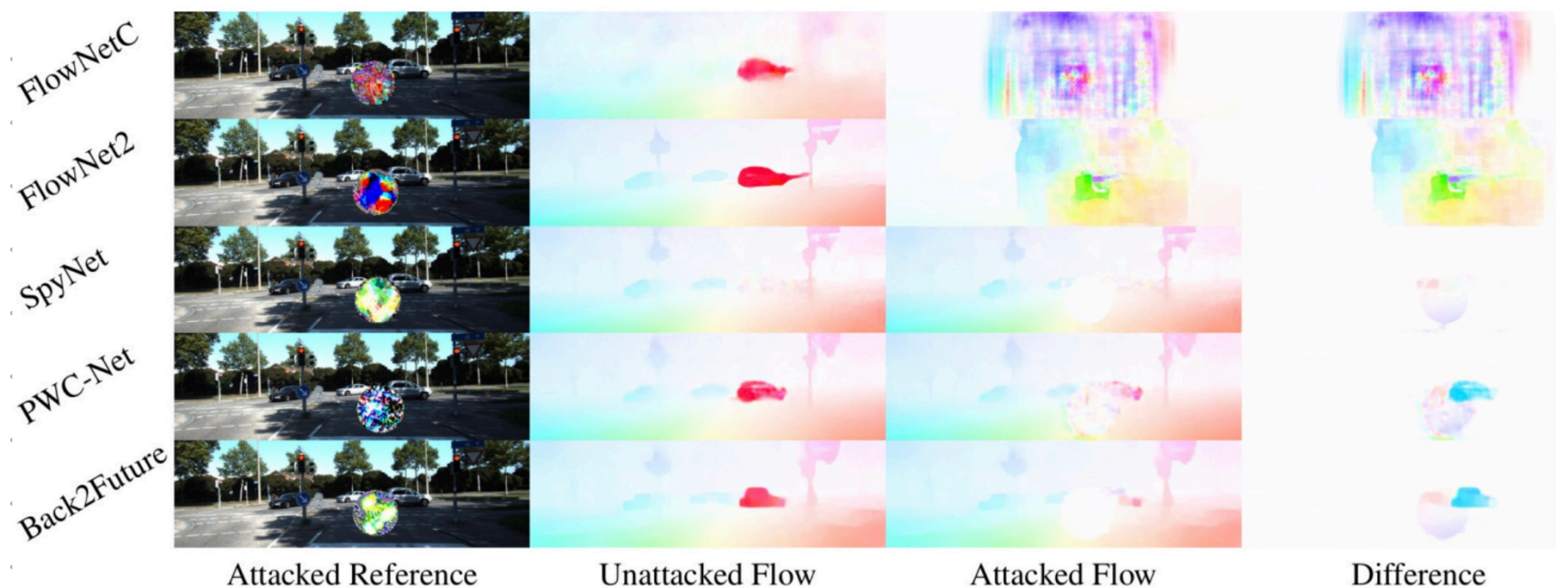
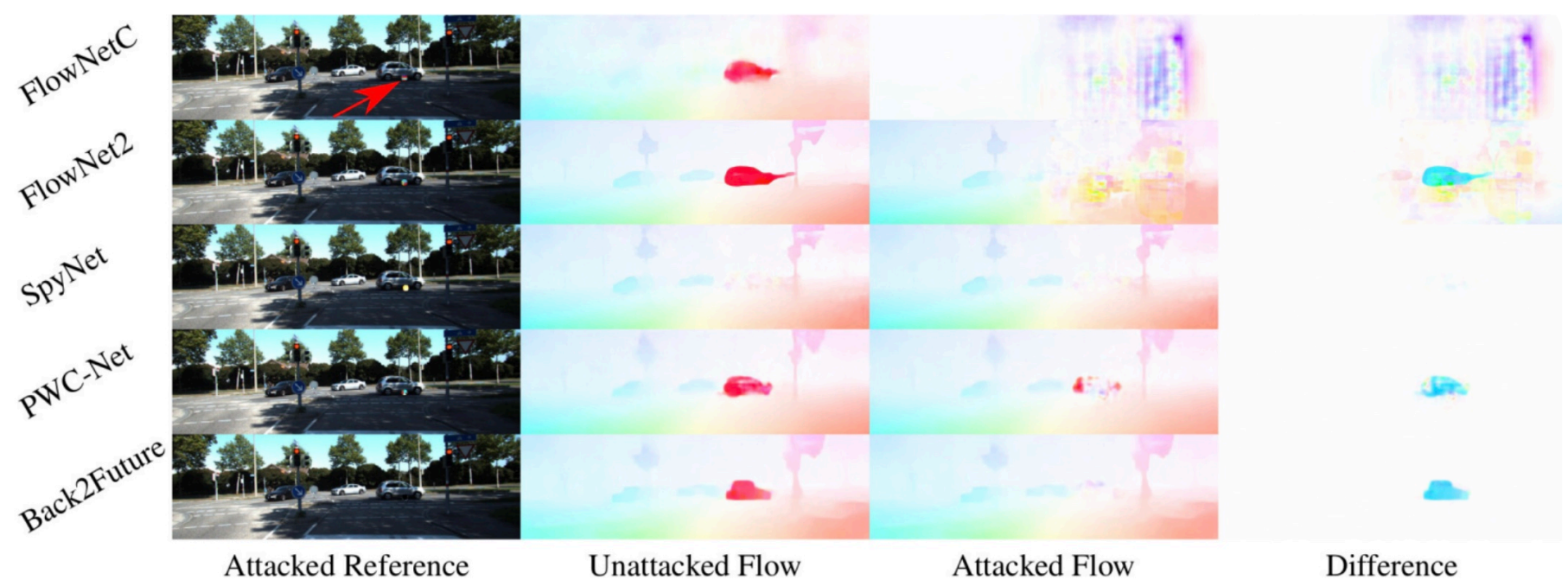
$$\hat{p} = \operatorname{argmin}_p \mathbb{E}_{(I, I') \sim \mathcal{I}, t \sim \mathcal{T}, l \sim \mathcal{L}} \left[\frac{(u, v) \cdot (\tilde{u}, \tilde{v})}{\|(u, v)\|_2 \cdot \|(\tilde{u}, \tilde{v})\|_2} \right]$$

unperturbed flow field
perturbed flow field

At any location in the image
under any transformation

with $(u, v) = F(I, I')$
 $(\tilde{u}, \tilde{v}) = F(A(I, p, t, l), A(I', p, t, l))$

Intuition: Find patch $\hat{p}$ that reverses the direction of the optical flow. *at every pixel in the image, ideally.*



PWCNet) and is still able to disturb the prediction of unseen networks. This is called a Black-Box Attack.

We can make the patch larger & perturb a larger region.

Defenses against Adversarial Attacks

Adversarial Attacks created a whole separate field that deals to defend these attacks. A review from 2019 summarized these defenses into three distinct categories

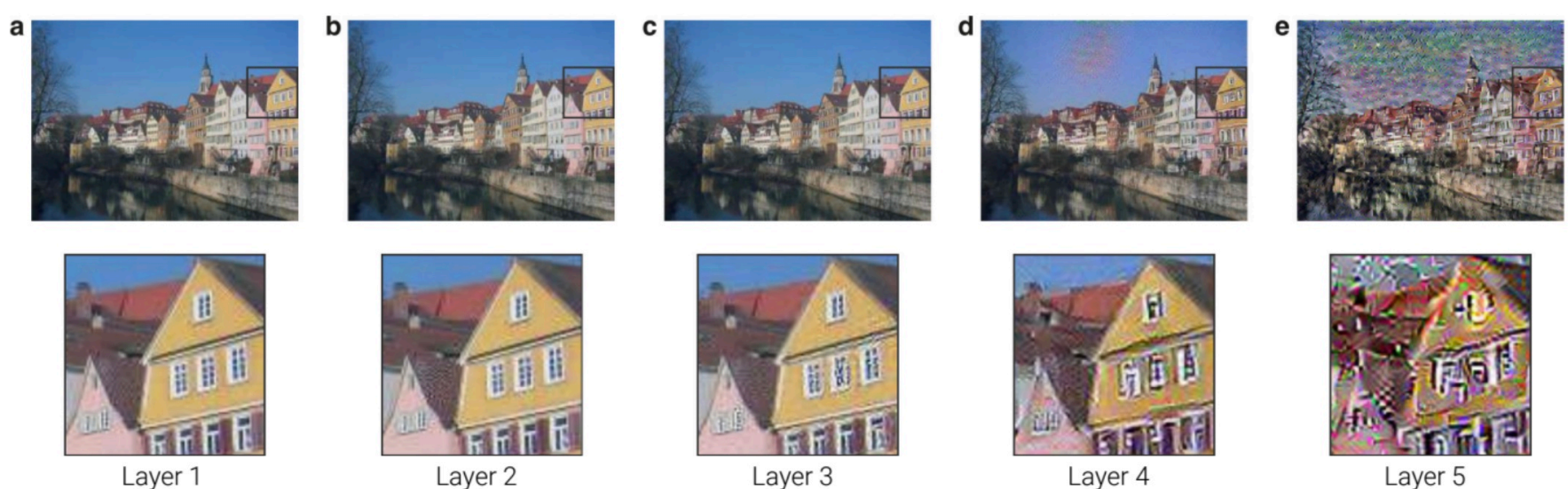
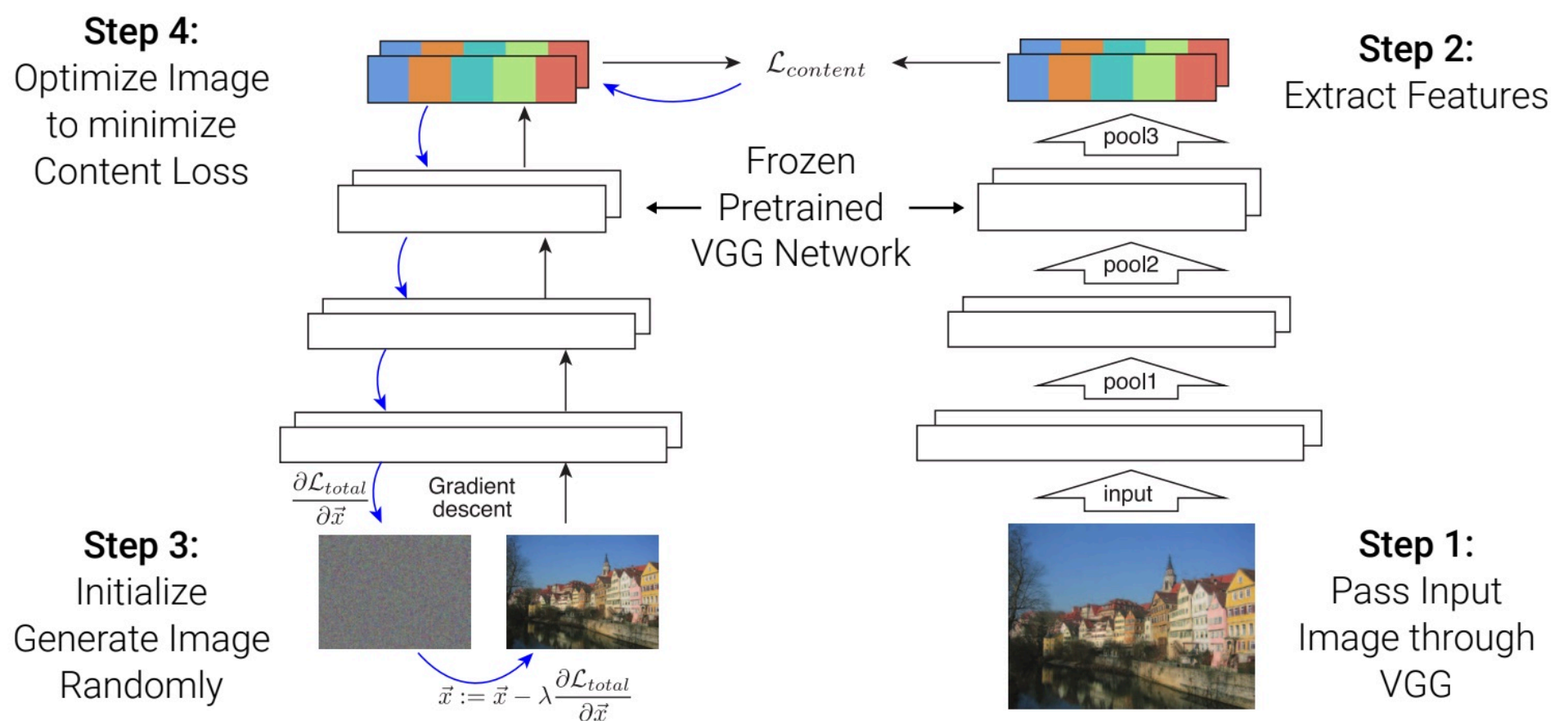
- **Gradient Masking/Obfuscation:** Hiding or masking the gradient of the model because most attack algorithms utilize the gradient information to exploit vulnerabilities of the classifier.
- **Robust Optimization:** A manner of learning, so that the model is more robust against adversarial attacks. By relearning and providing adversarial examples, the network thereby becomes more immune against constructed attacks.
- **Adversarial Example Detection:** Methods which first distinguish whether the input is benign or adversarial. Consequently, adversarial instances are rejected in the first place.

Neural Style Transfer

Synthesize an image with the content of one image & style of another

Start with random image, optimize content & style loss:

$$\mathcal{L} = \mathcal{L}_{\text{content}} + \mathcal{L}_{\text{style}}$$

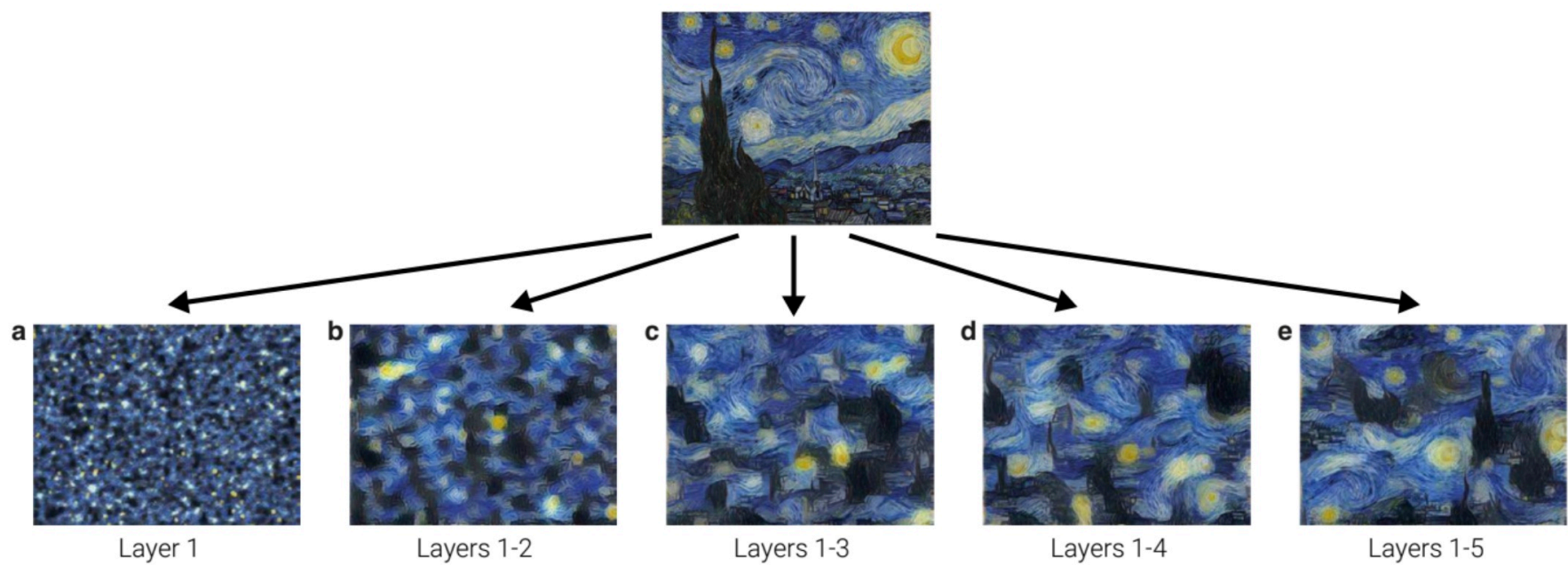


$$\mathcal{L}_{\text{content}}(l) = \|\mathbf{F}_l - \mathbf{P}_l\|_2^2$$

► $\mathbf{F}_l$: Features of generated image at VGG layer l

► F_l : Features of **generated** image at VGG layer l

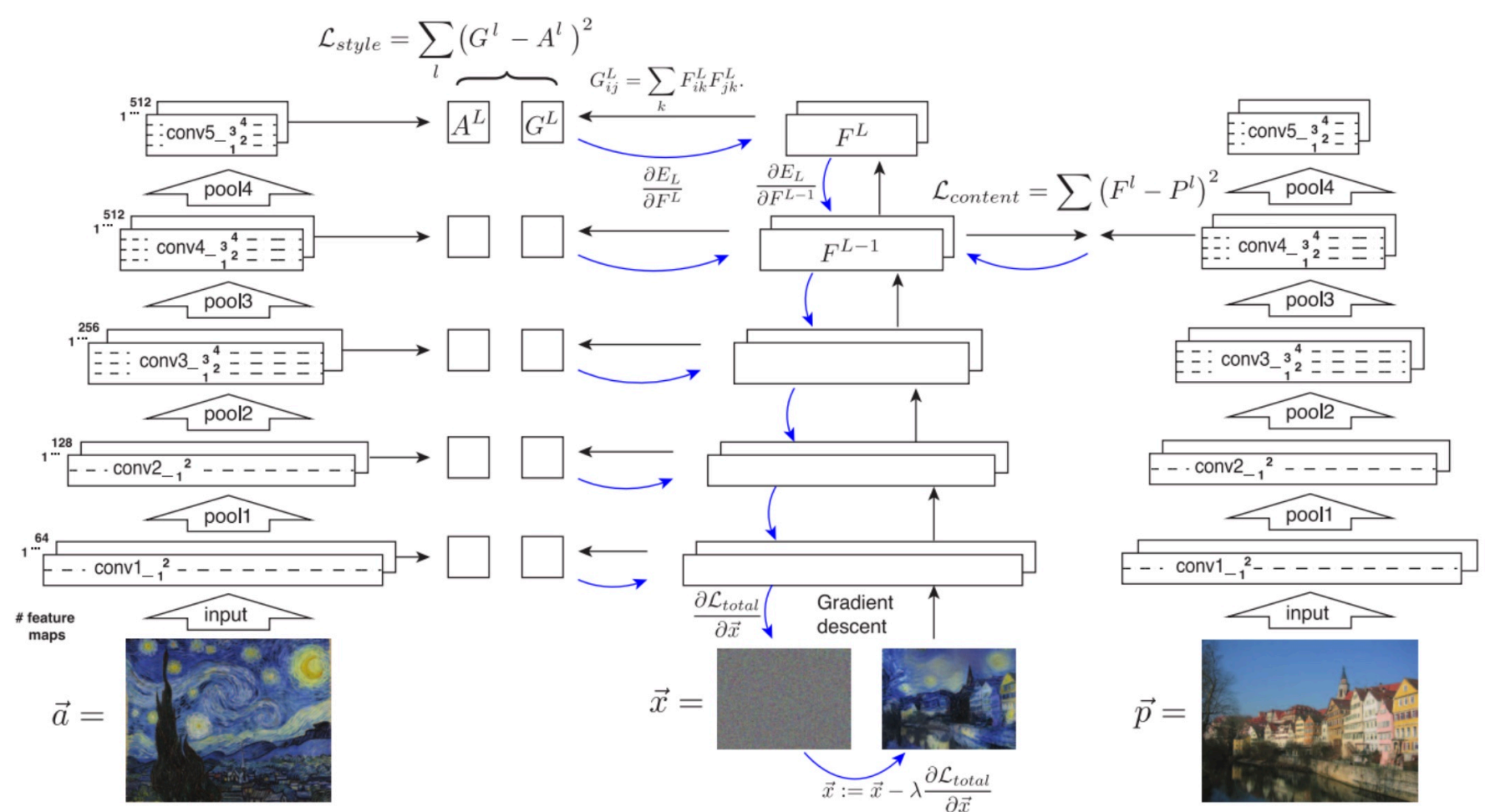
► P_l : Features of **content** image at VGG layer l



$$\mathcal{L}_{\text{style}}(l) = \|\mathbf{G}_l - \mathbf{A}_l\|_2^2 \quad G_{ij}^l = \sum_{\mathbf{p} \in \Omega} F_i^l(\mathbf{p}) F_j^l(\mathbf{p})$$

► $\mathbf{G}_l, \mathbf{A}_l$: **Gram matrices** (feature correlations) of generated and style image

Style loss is combined over several layers of the network. By using Gram matrices, the content of the style image becomes almost unrecognizable.



Lecture 12.2: Compositional Models

Shape Abstractions

Previous representations voxel, point clouds, meshes, implicit encode a large number of unstructured geometric elements i.e. there is no semantic information about the scene eg the individual parts or composition of objects.

3D primitives

Humans perceive an object as a composition of several atomic parts. As such, primitive based reconstruction is quite natural.

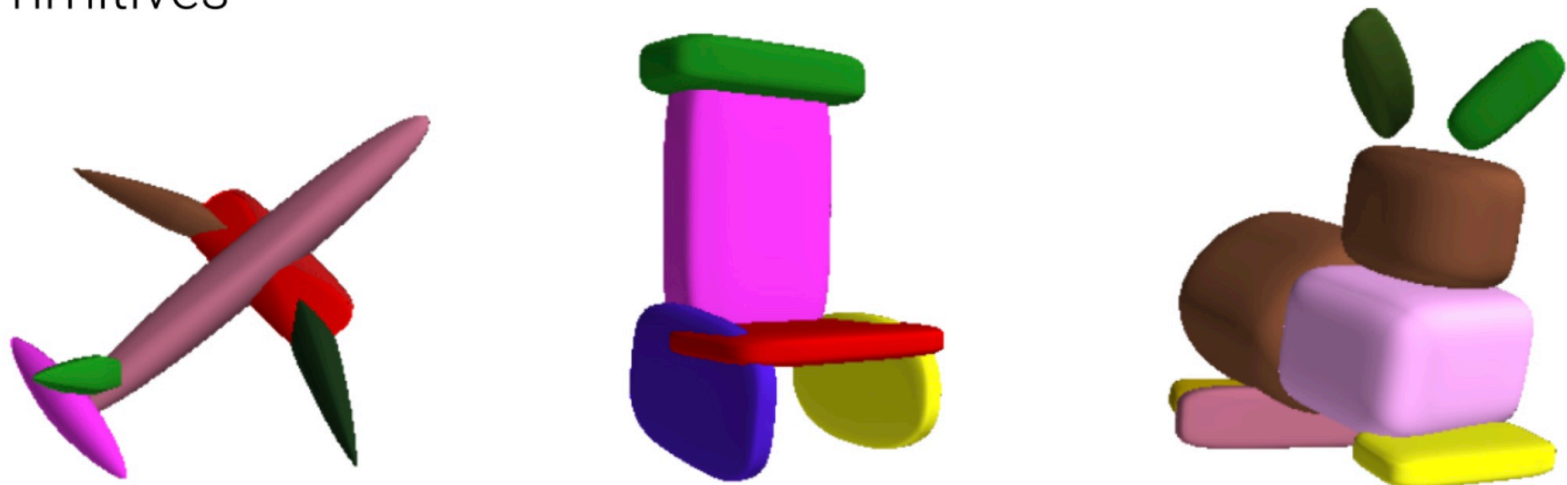
A 3D object encoded using a small no. of geometric shapes called primitives.

As we use few primitives, it is memory efficient but still manage to convey semantic information.

Nevertheless, there are challenges.

- there are variable no. of primitives i.e. no. of unknowns is unknown
- There are few annotated datasets & thus we need to learn without supervision.

Primitives



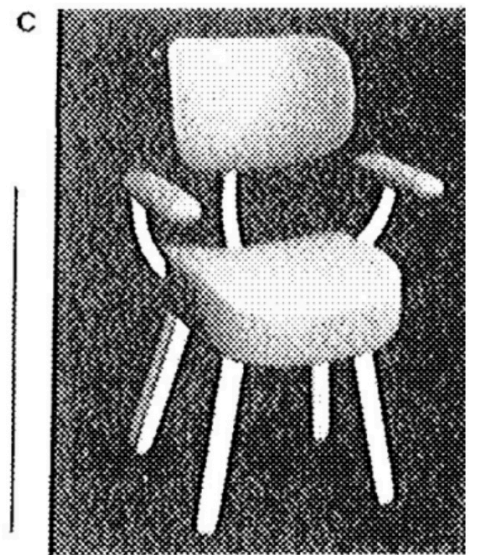
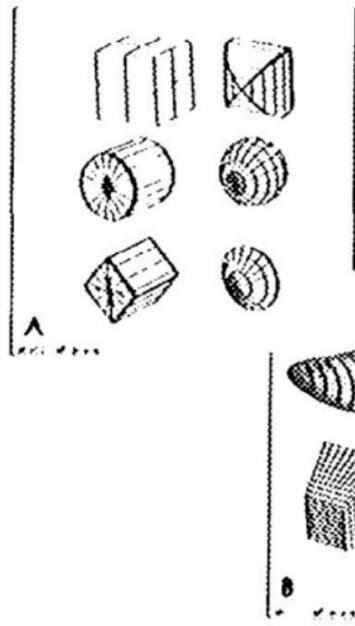
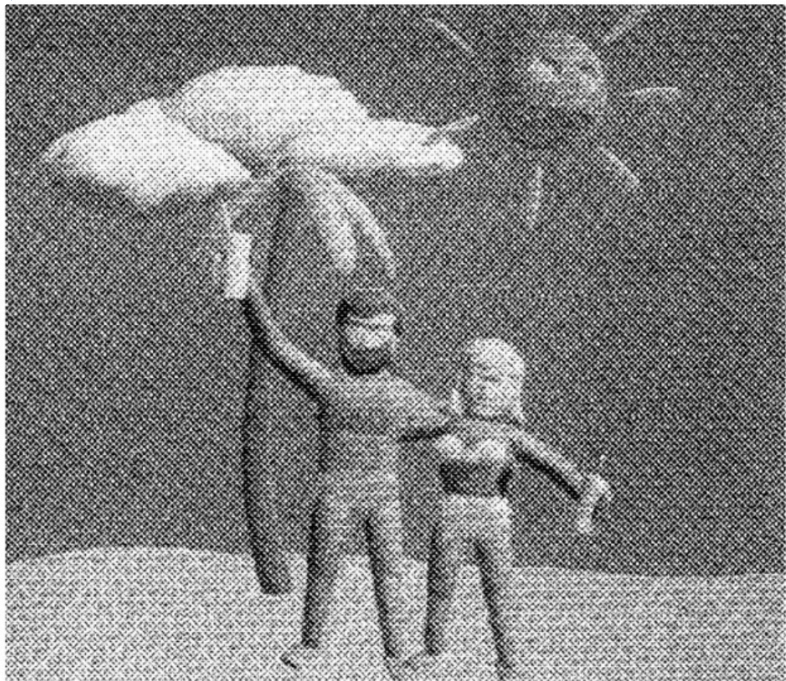
Goal:

Learn 3D shape primitives
Infer no. of primitives

No 'supervision' at primitive level i.e. we don't know which part is actually what.

Input: sparsely sampled point cloud

Pentland's Superquadrics Revisited

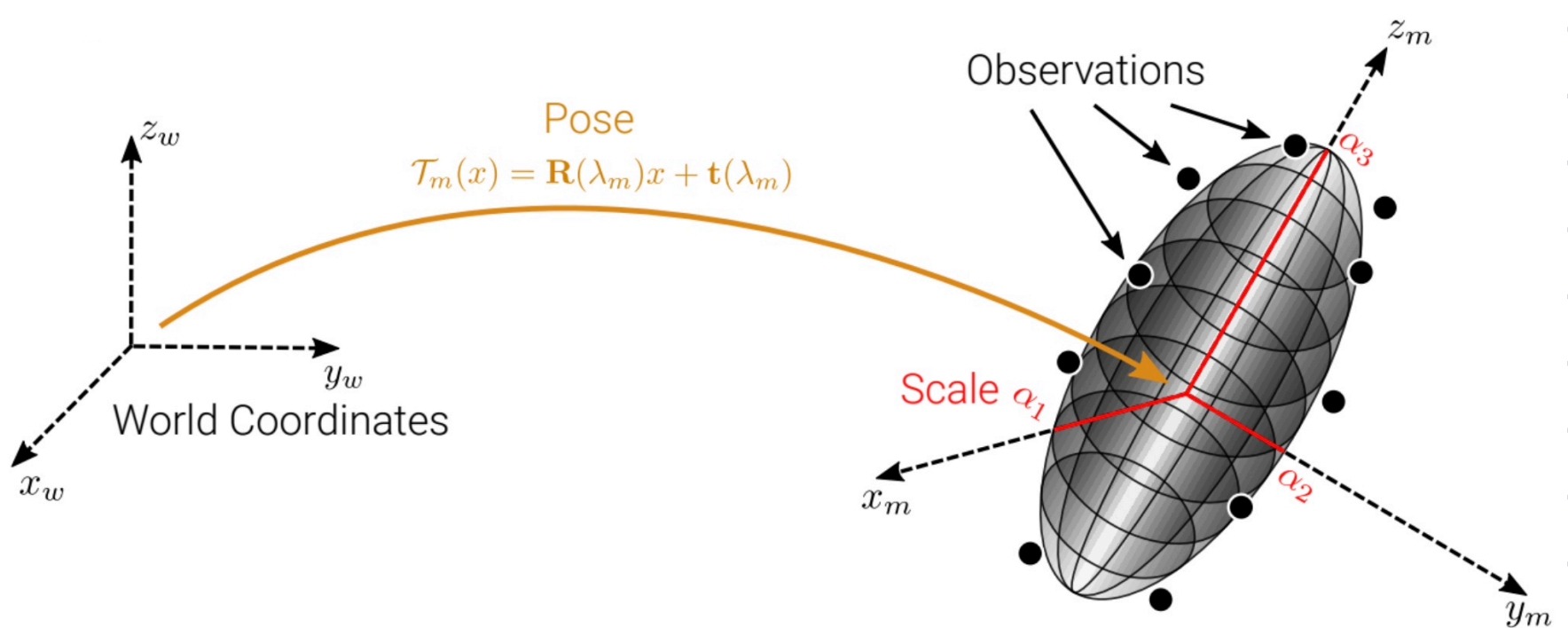


↳ stored in 1000 bytes

1 superquadric = 11 parameters

However, early approaches failed find these parameters.

Learning 3D Shape passing

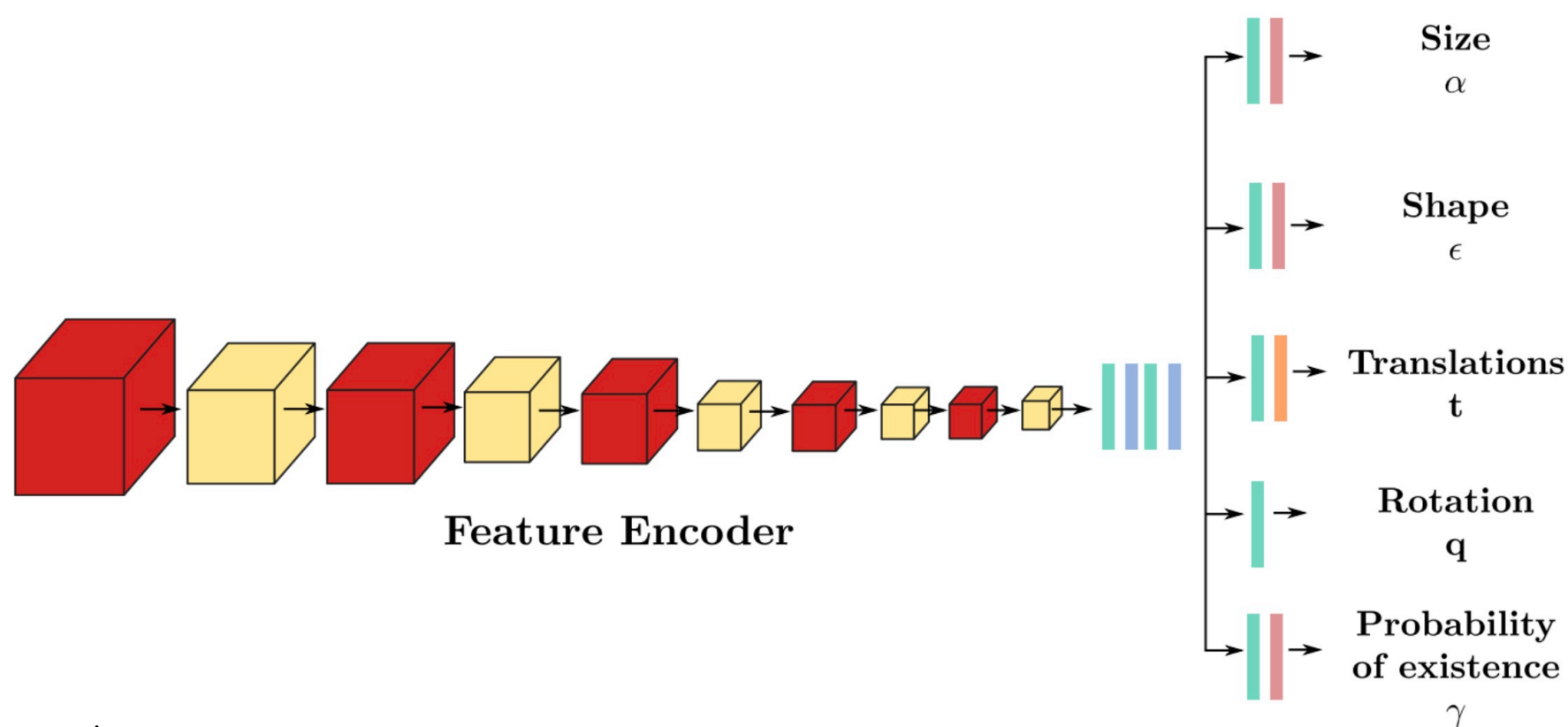


Predictions per primitive:

11 dof: 6 pose (R, t) + 3 scale (α) + 2 shape (ϵ)

Probability of existence

Network Architecture



Loss function

$$\mathcal{L}(\mathbf{P}, \mathbf{X}) = \mathcal{L}_{P \rightarrow X}(\mathbf{P}, \mathbf{X}) + \mathcal{L}_{X \rightarrow P}(\mathbf{X}, \mathbf{P}) + \mathcal{L}_{\gamma}(\mathbf{P})$$

$\mathcal{L}_{P \rightarrow X}(\mathbf{P}, \mathbf{X})$: Distance from predicted primitive to point cloud

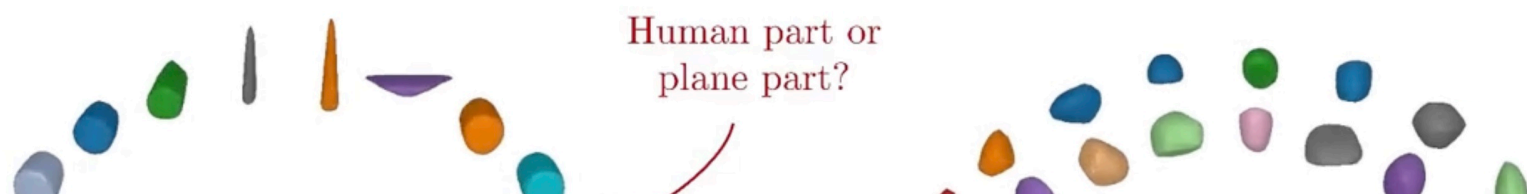
$\mathcal{L}_{X \rightarrow P}(\mathbf{P}, \mathbf{X})$: Distance of point cloud to predicted primitive

$\mathcal{L}_{\gamma}(\mathbf{P})$: Existence (at least one primitive) $\downarrow$ Parsimony (sparsity) Loss
 Have few primitives

In practice, Loss is evaluated by sampling points on the surface of each super quadric. These points are used to approximate the expected loss

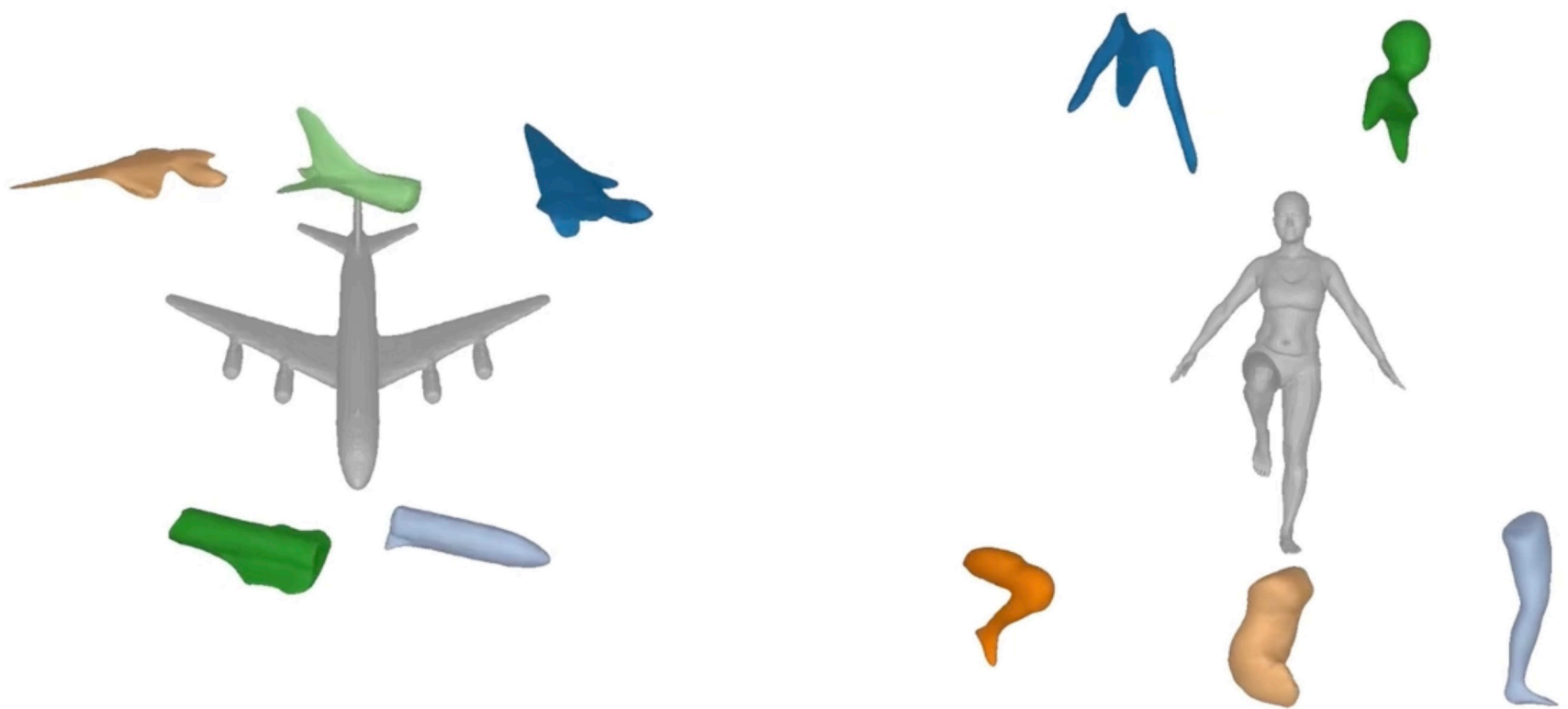
Neural parts

We need large number of parts to capture complex geometries, but if we use more parts, then they are less interpretable abstractions.





So, we use neural parts that are not limited to specific family of shapes.



Causal Reasoning

Neural networks learn spurious correlations & often take shortcuts.

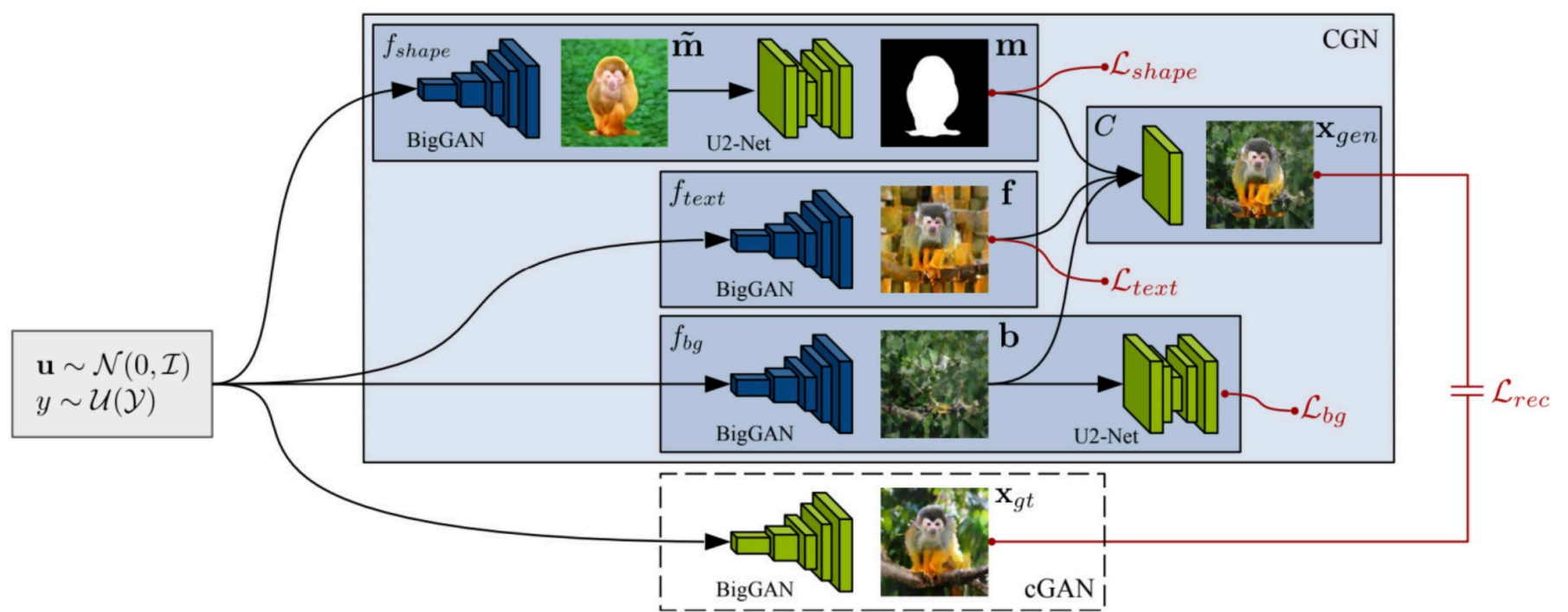
Can we learn to decompose image generations into causal mechanisms?

This allows us to ask counterfactual Qs:

"How would this image look like with different backgrounds?"

Training on counterfactual data increase classifier robustness to OOD.

Counterfactual Generative Networks



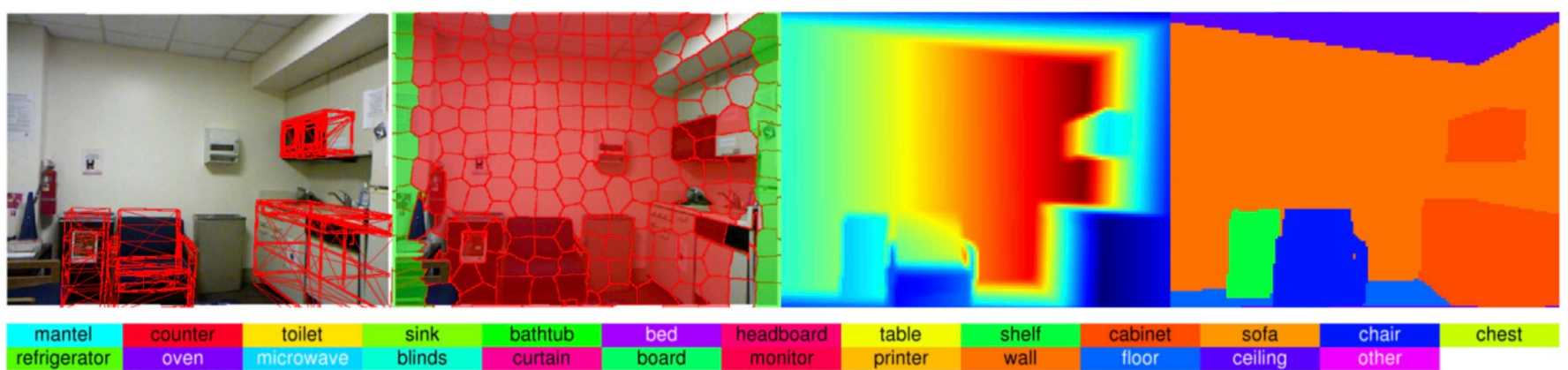
Shape	red wine	cottontail rabbit	pirate ship	triumphal arch	mushroom	hyena dog
Texture	carbonara	head cabbage	banana	Indian elephant	barrel	school bus
Background	baseball	valley	bittern (bird)	viaduct	grey whale	snorkel

Figure 4: **ImageNet Counterfactuals.** The CGN successfully learns the disentangled shape, texture, and background mechanisms, and enables the generation of numerous permutations thereof.

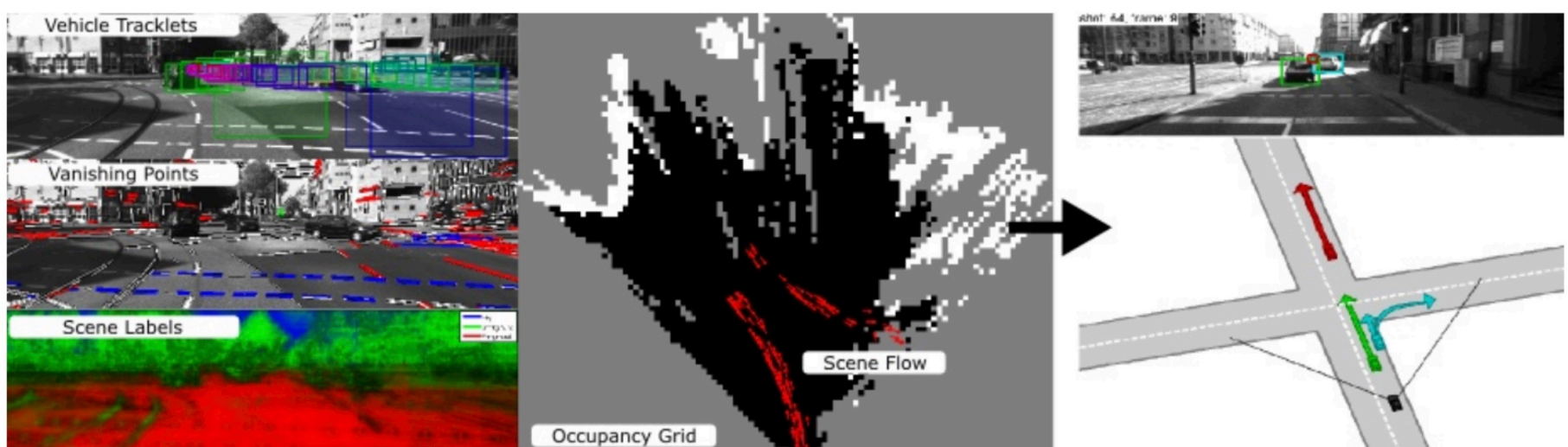
Holistic 3D Scene Understanding

Indoor scene understanding

Infer 3D objects & layout of indoor scenes from a single RGB-D image.



Urban scene understanding



- Multi-object **traffic scene understanding** from movable platforms
- Reason about the 3D scene layout, the location of objects and traffic activities

Lecture 12.3: Human Body Models

Humans interact through their bodies. Activities of humans are mainly defined by particular movements & gestures.

Generative Human Body Model

The goal is model a realistic looking minimally clothed human, as clothing is hard to model that moves & deforms like real people.

Ideally, the model can be fitted with real-world data, and be represented by small number of parameters

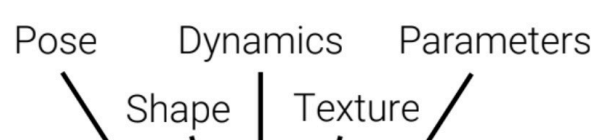


Early Body Models

Human body modelling has a long history in Computer Vision. However, the task is difficult due to variety of shape, physique and proportions of human bodies. SMPL is inspired by a morphable 3D face modelling method from 1999. In this method, a data set of 200 colored 3D images of faces is utilized to derive a single morphable reference face. As a result, a face can be represented by a vector space in relation to the reference face. Furthermore, the continuous parameters can be modified to change to perceived gender, fullness, or expression of the face.

A Skinned Multi-Person Linear Model

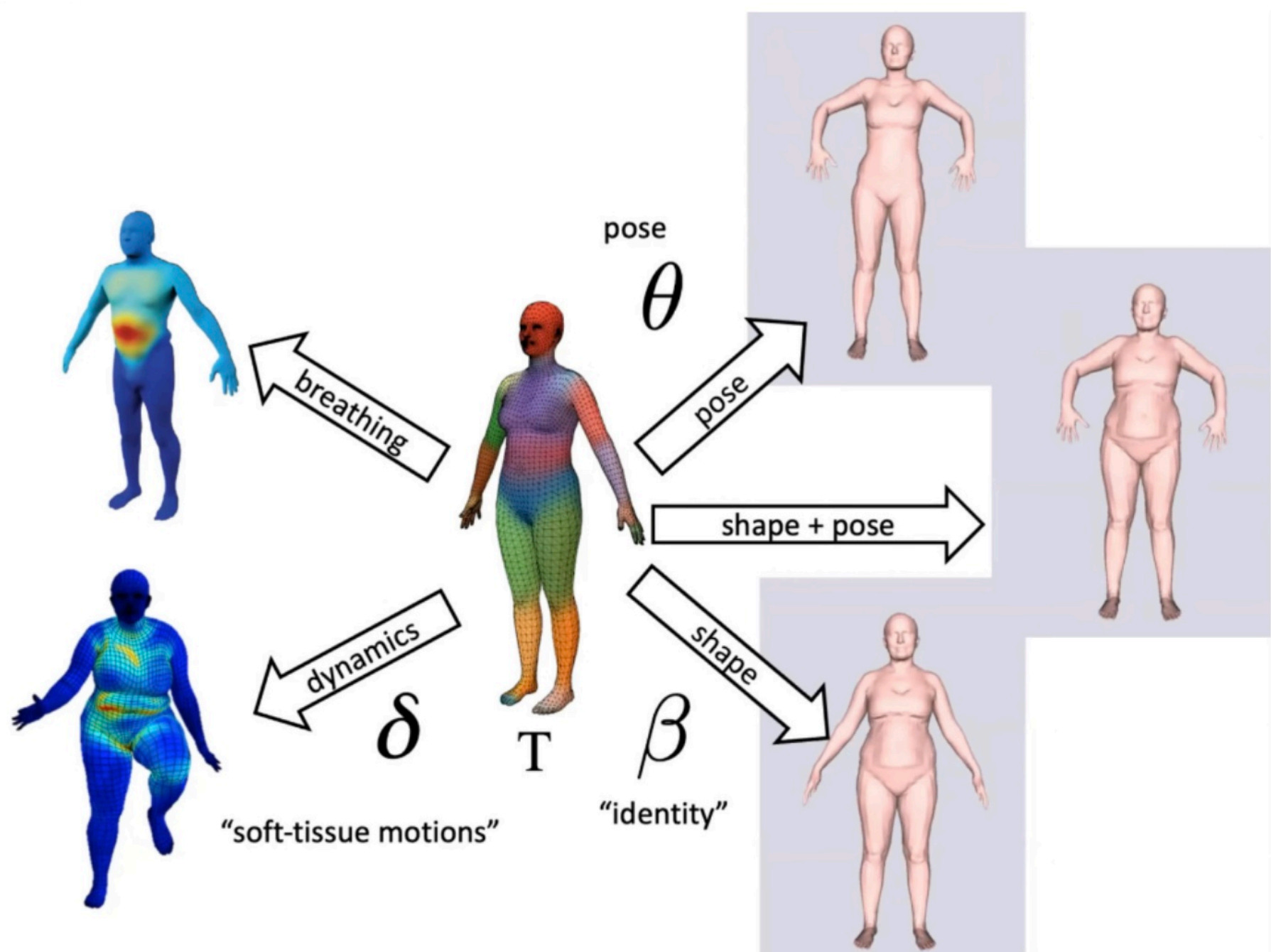
It utilizes thousands of human 3D body scans. The output human body is represented by a mesh



$$M(\vec{\theta}, \vec{\beta}, \vec{\delta}, \vec{A}; \vec{P})$$

↑
3D Mesh

We create the generative model in a way such that it factored, and we different controls for shape, dynamics, etc.



A reference mesh was designed by an artist to match a wide variety of human bodies & consist of 7000 vertices ($7000 \times 3 = 21,000$ numbers)

Most configuration of these numbers correspond not to people but noise.

Data Re-processing: Mesh Registration

Raw scans are unordered incomplete 3D points

The goal is to align/deform the reference body model for each scan.

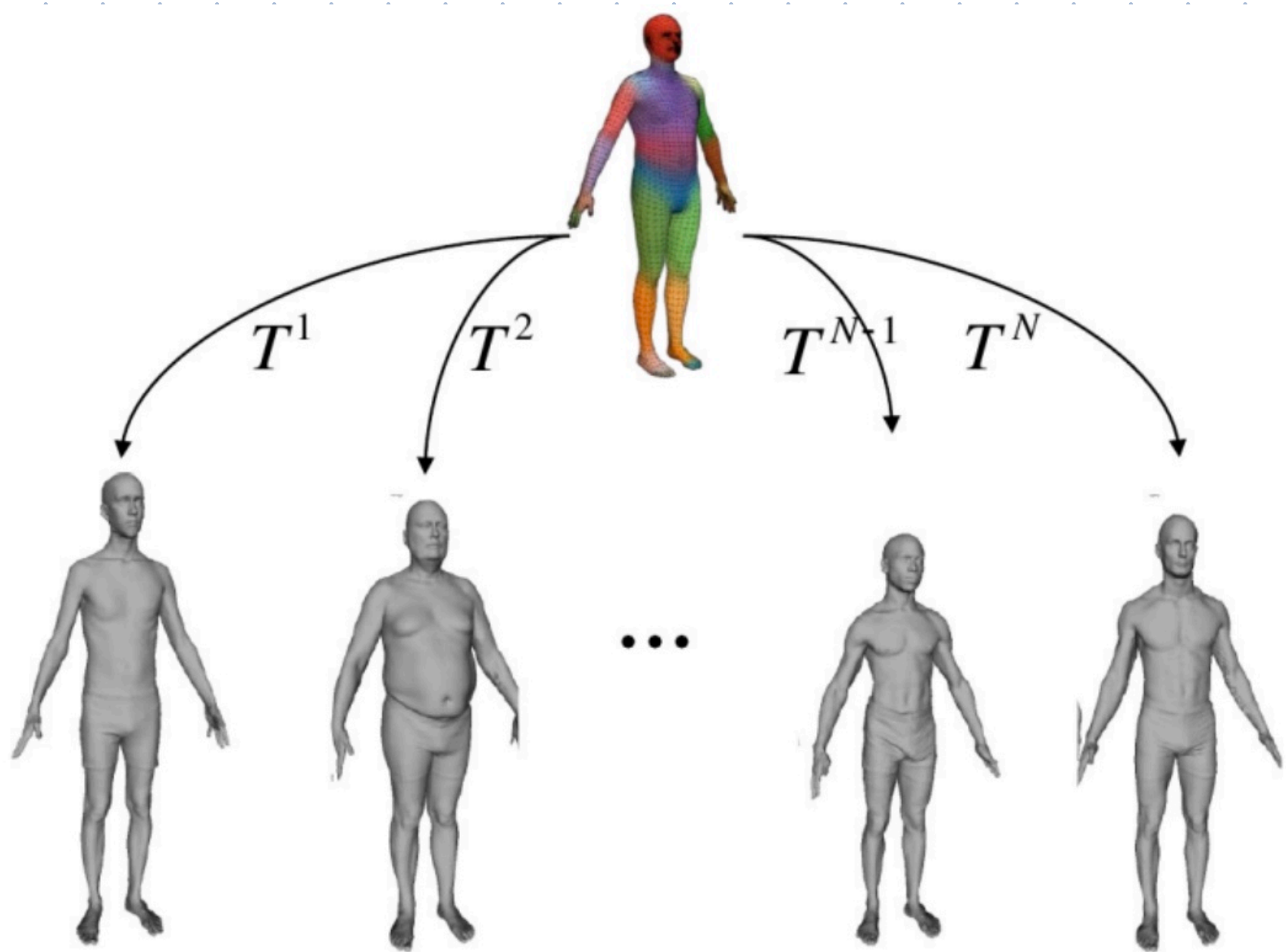
This is a chicken-and-egg problem as we need the trained model to deform the reference human body mesh but you need observations to train the model.

So, we jointly solve model & registration

Shape Deformation Subspace

Assume registration problem is solved.

Subsequently, the scan can be converted into a standard pose of each person, called "A pose".

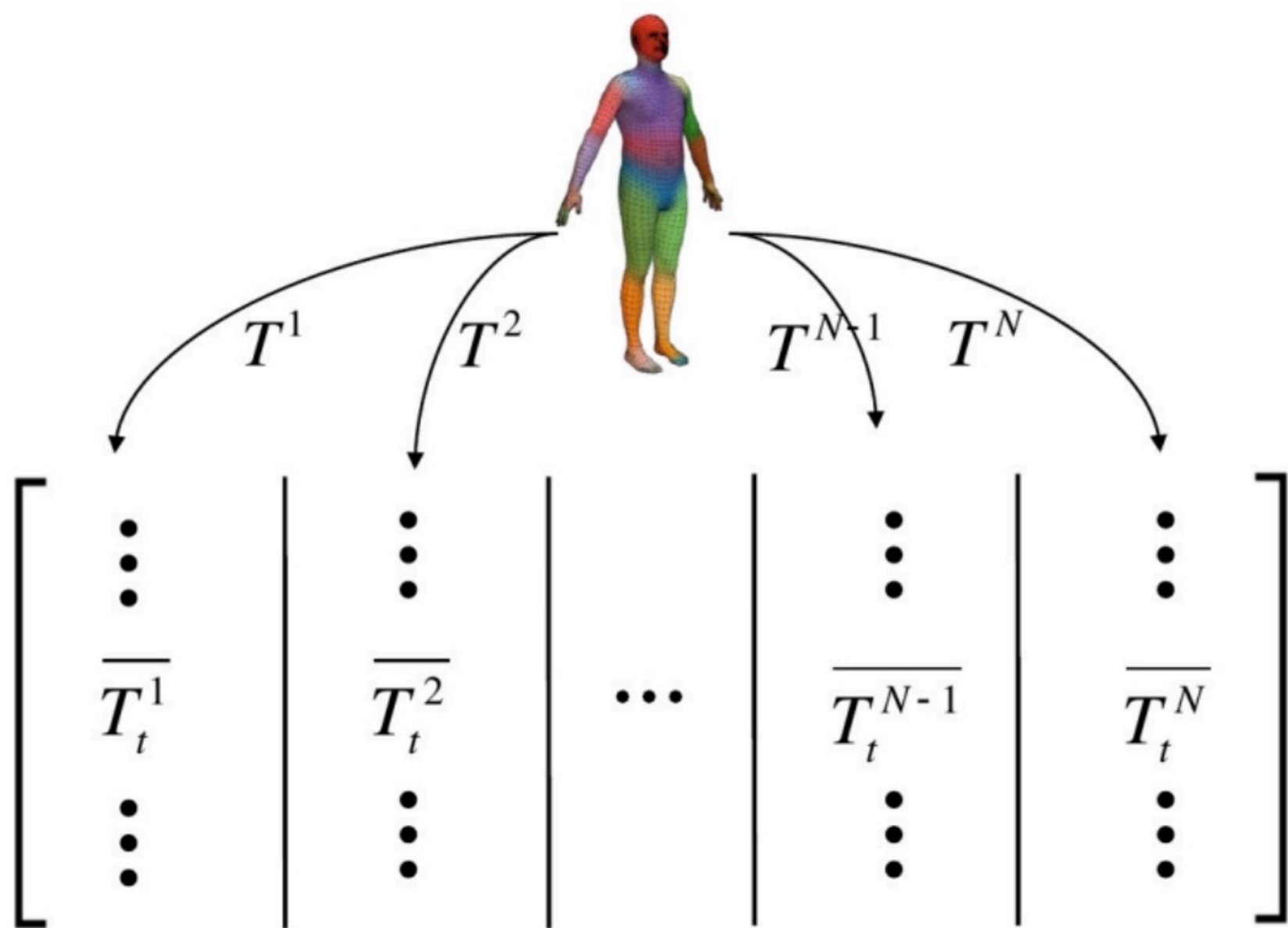


All mesh vertices are vectorized $\mathbf{T}$ & then subtract the mean mesh (colored)

These vectors are mapped into a large $\mathbf{T}$ matrix (with columns $\mathbf{T}^i$) via PCA.

$$\mathbf{T} = \mathbf{S}\boldsymbol{\beta} + \boldsymbol{\mu} \rightarrow \text{go from low-dimensional back to high dimensional by adding all PCs.}$$

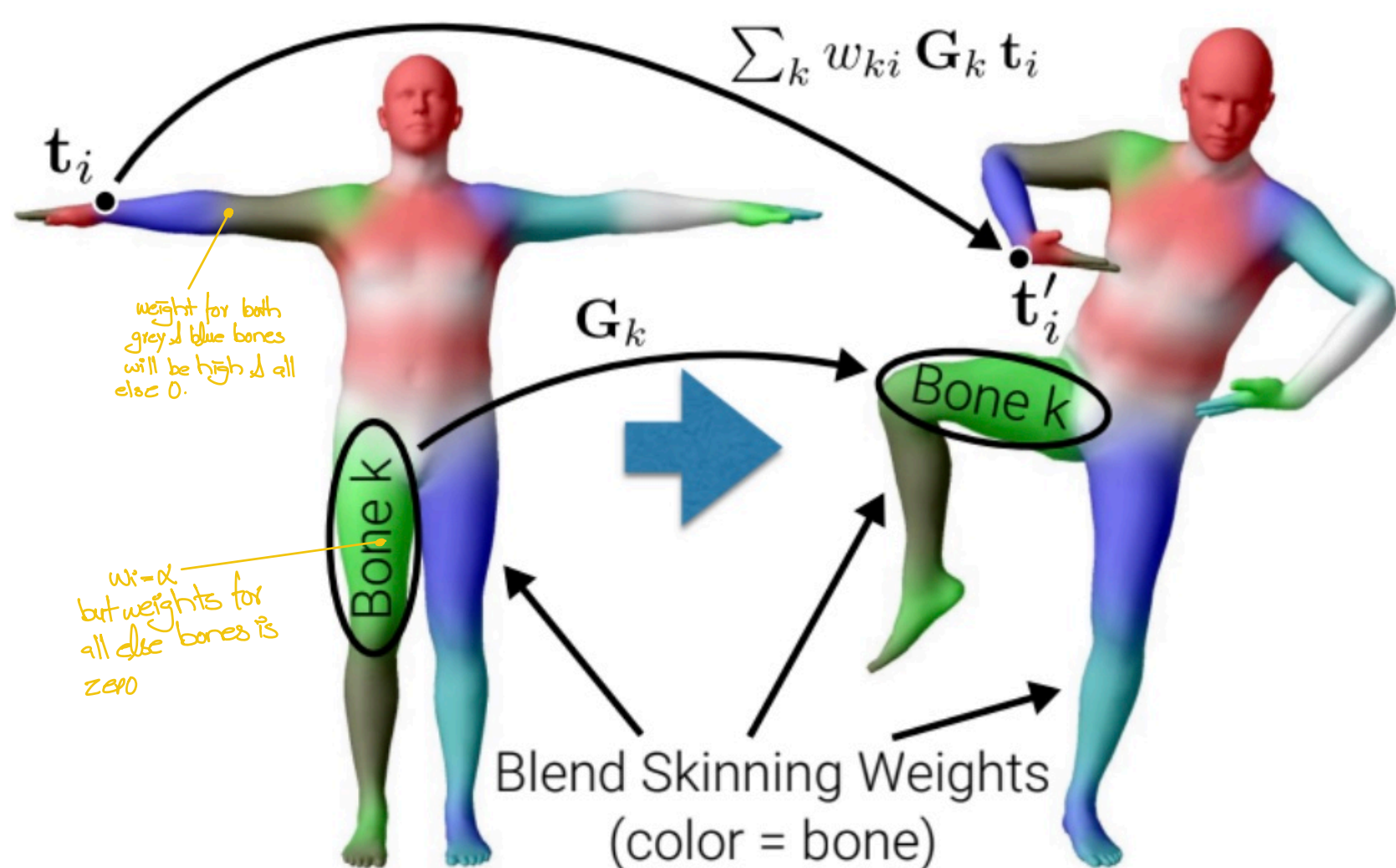
where PCA yields a linear model in a low-dimensional space. As a result, 21,000 is reduced to 10-300 dimensions.



Linear Blend Skinning (LBS)

SMPL also solves the mesh registration, and it does it through LBS.

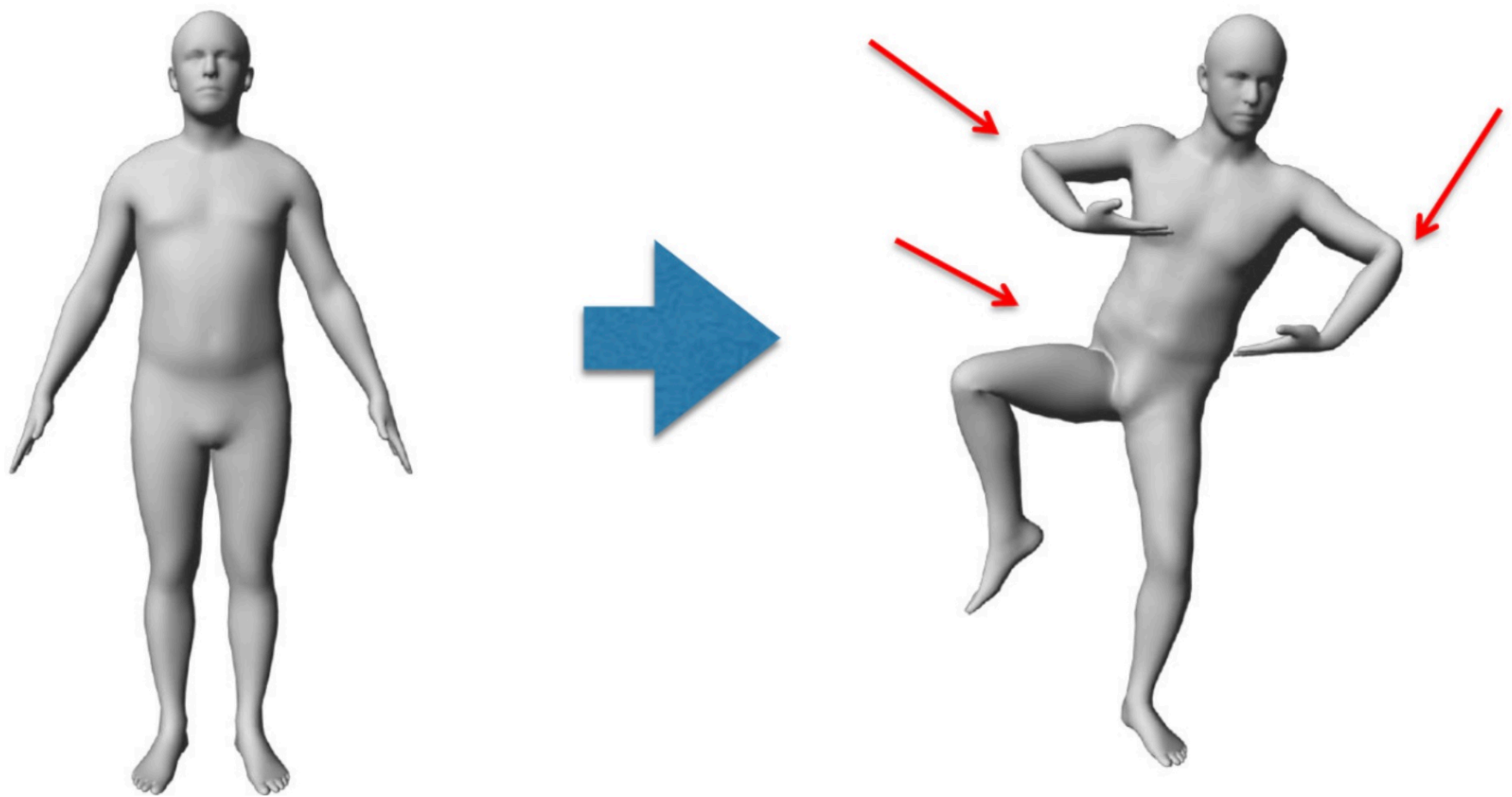
The goal is to for each vertex on the reference mesh, find the transformation to vertex in target pose.



$$\mathbf{t}'_i = \sum_k w_{ki} \mathbf{G}_k(\theta, \mathbf{J}) \mathbf{t}_i$$

- ▶ $\mathbf{t}, \mathbf{t}'$: Rest/transformed vertices
- ▶ w_{ki} : Blend skinning weights → how much each bone influences the transformation
- ▶ $\mathbf{G}_k$: Rigid bone transformation → changes a bone's position & orientation without changing its shape
- ▶ θ : Pose
- ▶ $\mathbf{J}$: Joint locations
- ▶ Skinning weights created by artist

LBS Problems



So, SMPL does:

$$\mathbf{t}'_i = \sum_k w_{ki} \mathbf{G}_k(\theta, \mathbf{J}(\beta)) (\mathbf{t}_i + \mathbf{s}_i(\beta) + \mathbf{p}_i(\beta))$$

Joint $\mathbf{J}(\beta)$ depends on body shape β i.e. there are tall people, short people & thus bone length, location of joints be adjusted.

Shape $\mathbf{s}_i(\beta)$ & pose $\mathbf{p}_i(\beta)$ correctives are added to $\mathbf{t}_i$. Shape correctives allows the model in rest pose to accommodate different shapes i.e. heavy vs thin people.

Pose corrective allows to accommodate pose θ .

Dynamic SMPL

focused on realistic soft-tissue dynamics for body shape & motion, in particular for heavy people.

SMPLify:

A human body model is estimated from single image.

Lecture 12.4: Deepfakes

Deepfakes focus on swapping face in photos & videos.

Face Swap

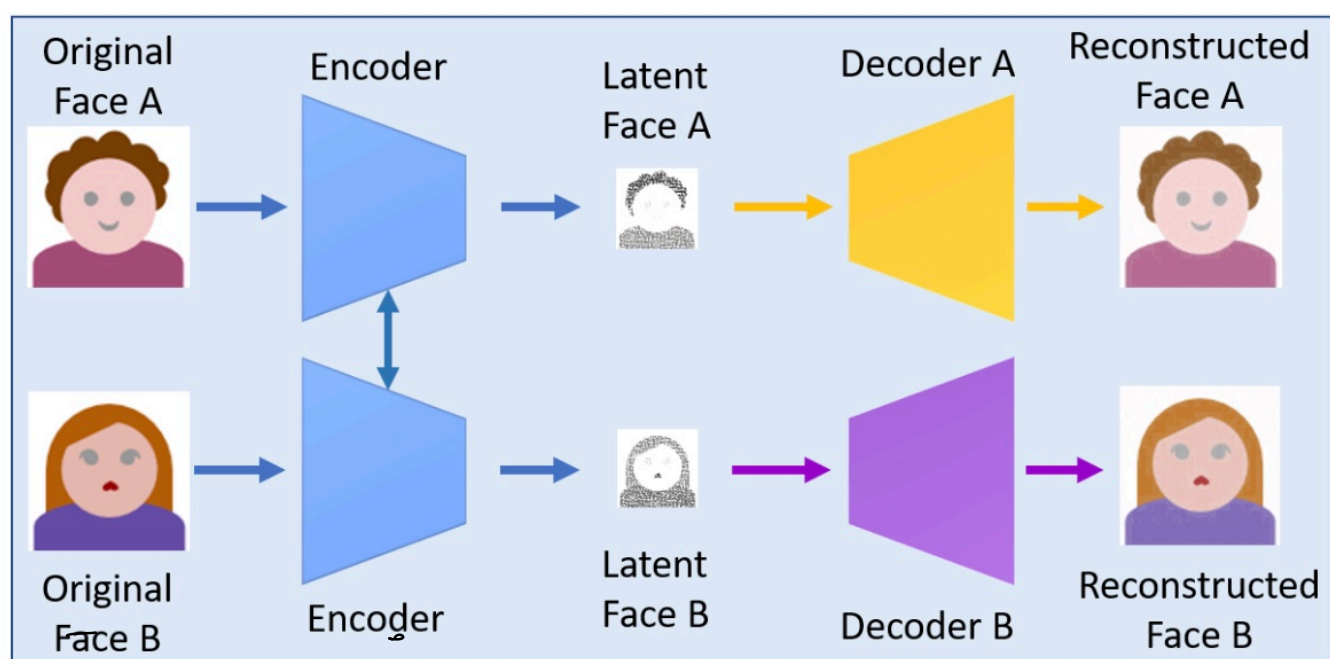
This model uses an auto-encoder, which was trained on the footage and faces of two people.

The encoder is shared, which is used to extract latent features of both persons.

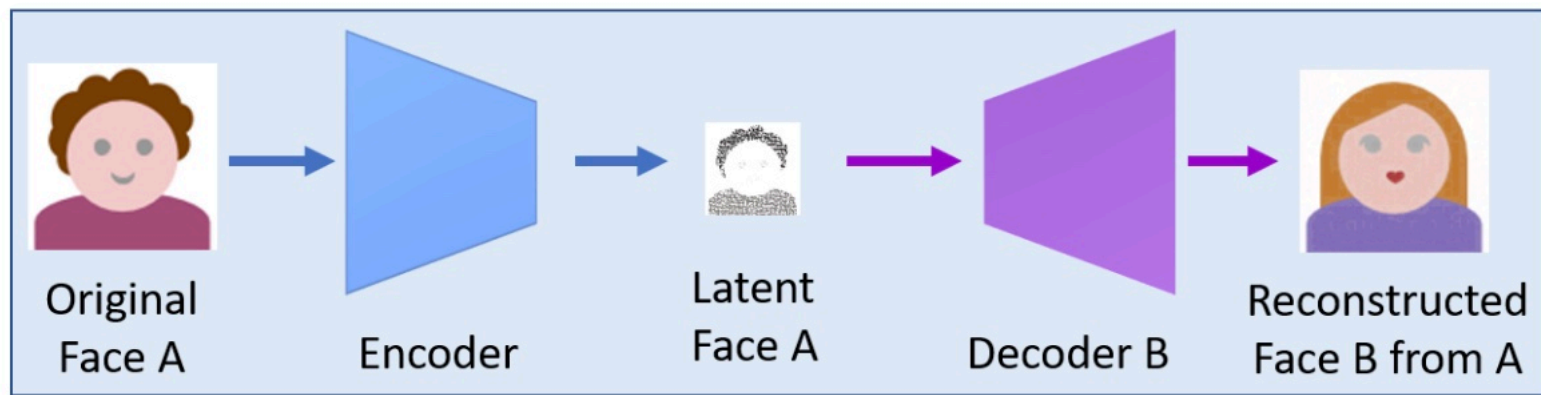
A separate decoder is trained for each face individually to reconstruct the footage from the latent space.

At inference time, we swap the decoder, so that the face of a person is reconstructed with appearance of another.

Train time.

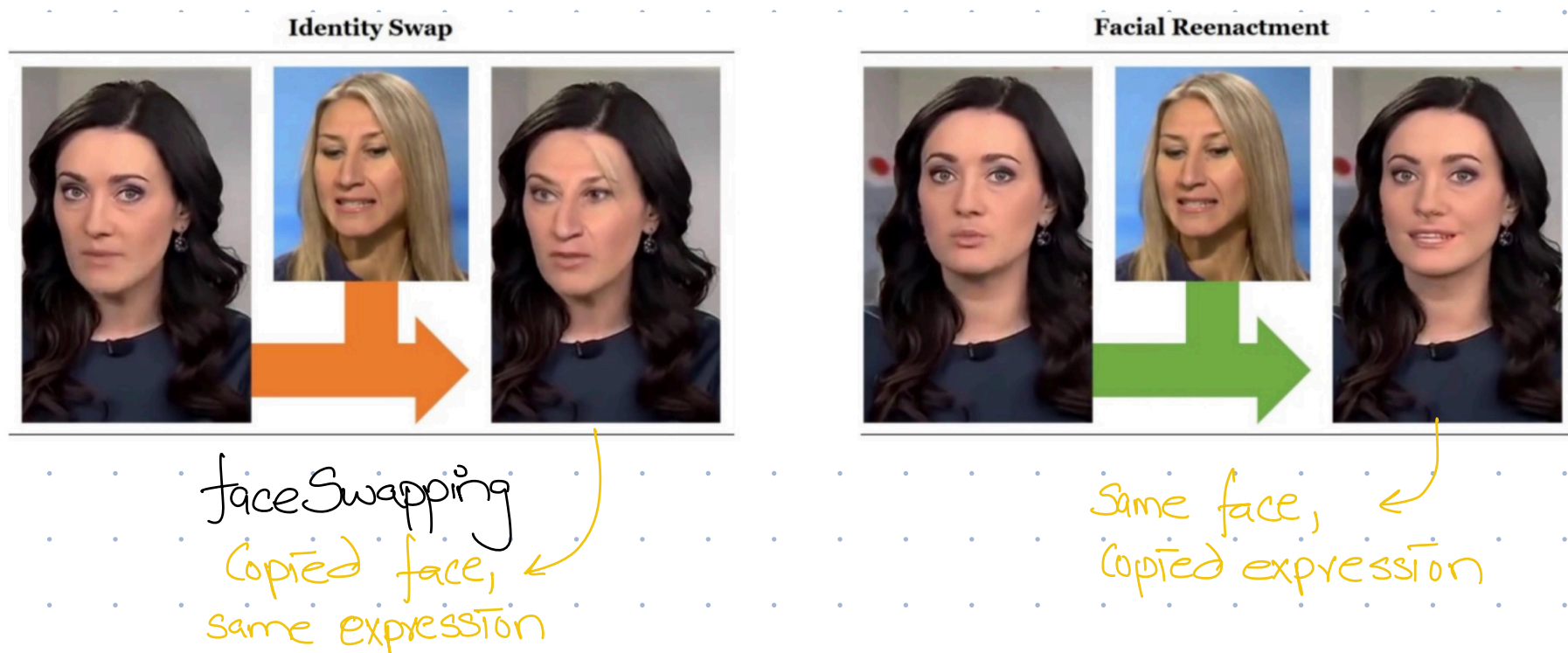


Inference time:



Reenactment

Facial reenactment takes an expression from an image or a video & animates it on another person.



Face 2 Face

The goal is to capture the facial expression from a source RGB-input and map it on a person in a target video.

All of this is done in real-time with a commodity webcam for the source input.

The method uses a face model to track the expression of the faces in the target & source video.
→ recovered like body models above

The parameters of this face model are recovered by dense non-rigid alignment.

The expressions in the target & source video are tracked which enables the transfer of facial

expressions.



Dense non-rigid alignment

1. Non-Rigid (Expression and Deformation)

The term **non-rigid** means the objects being aligned are **deformable**. A *rigid* transformation would only allow the object to be rotated, scaled, or translated (moved) without changing its internal shape—like a skull or a block of wood.

- **Correct:** You are right that non-rigid in this context means **expression** is involved. The face is a non-rigid surface because its shape changes constantly due to muscle movement (smiling, frowning, speaking).
- **Goal:** The *non-rigid alignment* must find the specific deformation (a **deformation field**) that maps the neutral face model to the current expressive face in the video. The goal is to track the expression, which is represented by these localized, non-rigid changes.

2. Dense (Every Point/Pixel)

The term **dense** means that the alignment attempts to establish a correspondence for a **large number of points** (often every pixel or every vertex on a 3D model) on the surface, rather than just a few key landmarks.

- **Correct:** Your belief that it aligns all the points on the surfaces is accurate.
- **Contrast:** This is in contrast to **sparse alignment**, which only tracks a few hundred key facial landmarks (like the corners of the eyes or mouth).
- **Goal:** *Dense alignment* is necessary to capture subtle details like skin wrinkles, cheek bulges, or the exact shape of the mouth interior, which are critical for **photo-realistic** and convincing reenactment.

Deferred Neural Renderers

It uses a Neural Renderer, essentially a U-Net, which takes a feature representation & renders the output through it.

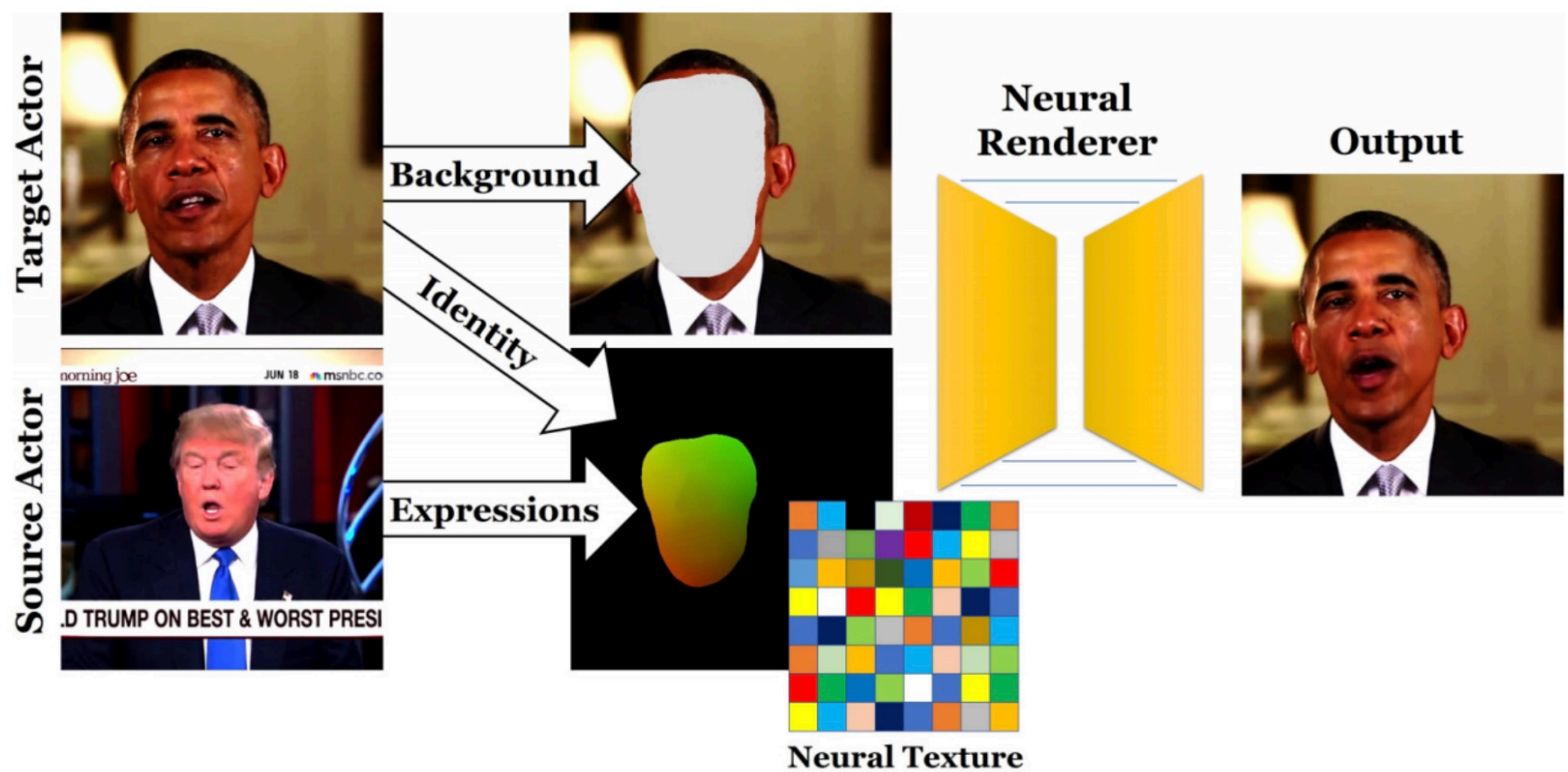
This allows to compensate for inaccuracies that model-based approach (above works) might have.

For example, the face model might not be registered

perfectly, as the face might not adhere to the face model perfectly, or it might be difficult to transfer some pixels due to occlusion.

To solve them, you need to hallucinate content, which this does.

This approach extracts face geometry & neural texture, where for each point we have a feature vector that captures parts of the appearance that should be on that surface point.



In short,
recovers geometry / face model
compute neural texture, which is a learned feature vector for every point on the face model.

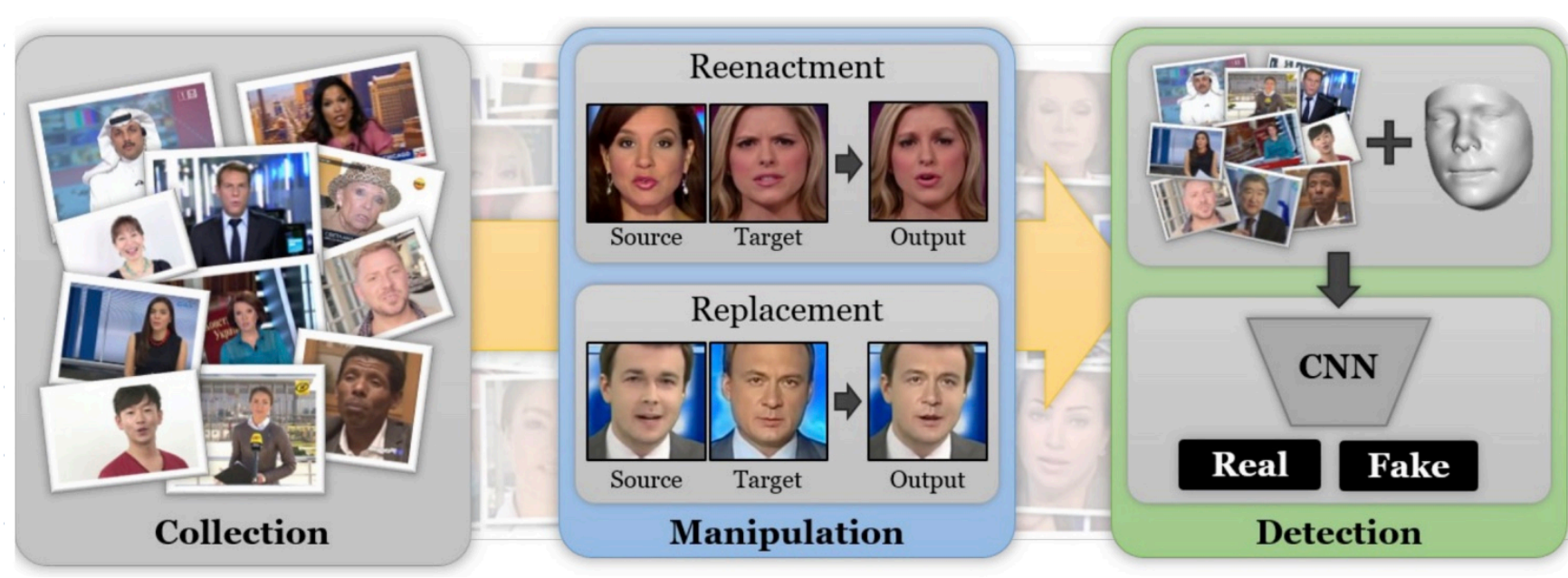
Pass these to Neural Renderer to reconstruct the image/video.

Deepfake Detection

Given a video, detect if a video has been artificially manipulated.

FaceForensics

Learn a binary classifier to detect manipulated images

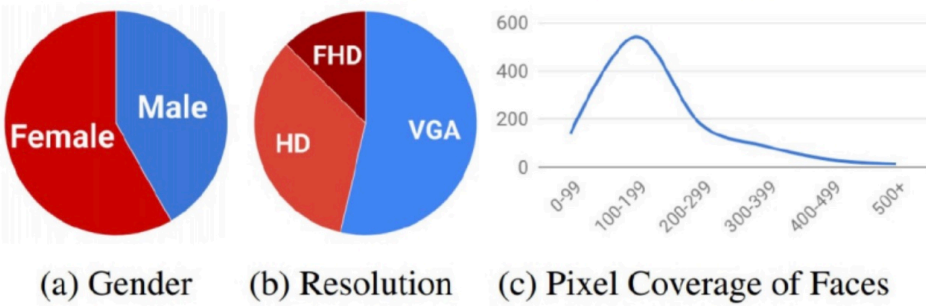


FaceForensics Dataset

Source: 1,000 Videos (510,529 frames)

Methods	Train	Validation	Test
Pristine	366,847	68,511	73,770
DeepFakes	366,835	68,506	73,768
Face2Face	366,843	68,511	73,770
FaceSwap	291,434	54,618	59,640
NeuralTextures	291,834	54,630	59,672

- Publicly available!
- Over 2 million manipulated frames
- Three compression levels for each manipulated frame
- Over 1000 research groups



ForensicTransfer

XceptionNet	Test on Face2Face	Test on FaceSwap
Trained on Face2Face	98.13%	50.20%
Trained on FaceSwap	52.73%	98.30%

Conclusions:

- DeepFake detection works well on in-domain data (i.e., knowing the forgery method)
- However, it doesn't generalize
- Requires self-supervised, transfer and few-shot learning techniques for OOD generalization